



OWA Full-Stack Developer

Real-Time Event Tracking Platform

Time estimate: 3 days

Runtime constraint: The embeddable scripts must be written in Vanilla JS. No jQuery, React, or other libraries are allowed in any code that runs on the publisher's page.

Overview

Your task is to build a lightweight real-time event tracking system that can be embedded on any third-party website — similar in shape to a product analytics SDK. The system has three parts: an Event Backend, an Embeddable Tracking Runtime, and a Monitoring Dashboard.

Part 1 — Event Backend

Build an API server that handles event ingestion, session tracking, and widget delivery.

Storage

Use any SQL database. A schema.sql file with your table definitions is a required deliverable.

Endpoints

GET /config/:clientId

Returns the initialization configuration for a given client. At minimum: flush interval (how often the widget batches and sends events), and which event types to track.

GET /widget.js

Serves the compiled widget bundle.

POST /events

Receives a batch of events. The request body is a JSON array. Each event has at minimum: clientId, sessionId, eventType, timestamp, and an optional metadata object.

GET /sessions

Returns all sessions that have emitted at least one event in the last 5 minutes, with their event counts.



GET /stats/:clientId

Returns aggregate event counts broken down by event type for the last 5 minutes.

CORS

The backend must allow cross-origin requests from any origin so the widget can reach it from publisher pages.

Part 2 — Embeddable Tracking Runtime (Vanilla JS)

The Loader (loader.js)

A small snippet a customer pastes onto their site. It must:

- Not block the host page's rendering
- Fetch the widget bundle from your server asynchronously
- Initialize the widget using config from /config/:clientId

The loader must also implement the **queue pattern** for early calls: code on the host page may call the tracking API before the bundle finishes loading. The loader must buffer those calls and replay them once the widget is ready. Do not attach the tracking API directly to window — see the global scope rule below.

The Widget

Once loaded, the widget must:

1. Track Events Automatically

- Fire a pageview event on load
- Fire a click event on any element that has a data-track attribute, including the attribute's value as metadata
- Batch all events and flush them to POST /events on the interval from the server config — not on every event

2. Expose a Custom Event API

Provide a way for host-page code to fire custom events (e.g. `_tracker.push(['track', 'signup', { plan: 'pro' }])`). The exact API shape is your call.

3. Render a Live Viewer Badge

Inject a small UI element showing the active session count from GET /stats/:clientId, refreshed on the same flush interval. This element must be completely isolated from the host page's CSS — the host page's styles must not affect its appearance.



4. Implement destroy()

The widget must expose a destroy() method that removes all injected DOM nodes and cancels all timers and event listeners. No trace of the widget should remain on the host page after calling it.

5. Not Pollute the Global Scope

The widget must not attach any functions, objects, or state to window (except the loader queue variable used before the bundle loads). All internal state must be encapsulated.

Part 3 — Monitoring Dashboard

A simple interface for monitoring widget activity. No framework requirement — use whatever is practical. No authentication required.

Session List

All sessions active in the last 5 minutes, showing session ID, event count, and time of last event. Auto-refreshes every 5 seconds.

Session Detail

Click a session to see its full chronological event history (type, timestamp, metadata).

Event Breakdown

A summary table showing total event counts by type for a selected clientId over the last 5 minutes.

Deliverables

File	Notes
schema.sql	Table definitions. Reviewed first.
README.md	Setup instructions, how to initialize storage, how to run backend and dashboard, explanation of the ingest write strategy
test-site.html	Static page with the loader snippet. Opening it in a browser must show a working, visually isolated badge.
Source code	Backend, widget runtime, dashboard